WoW Class & Spec Sentiment Analysis Dashboard
Technical Documentation

---

1. Overview

The WoW Class & Spec Sentiment Analysis Dashboard is a data-driven analytics system designed to evaluate player sentiment surrounding World of Warcraft retail classes and specializations.

The system ingests community discussion data, processes unstructured text into structured insights, and produces both analytical summaries and actionable design recommendations. The goal is to simulate how player feedback can be systematically incorporated into game balance and systems design decisions.

---

2. Objectives

The primary objectives of this project are:

- Analyze large-scale player discussion data
- Quantify sentiment across classes and specializations
- Identify recurring gameplay themes (e.g., damage, survivability, utility)
- Compare sentiment across roles (Tank, Healer, DPS)
- Detect high-risk or underperforming specs
- Generate actionable design insights and recommendations
- Present findings through dashboards and visualizations

---

3. System Architecture

Data Sources → Collectors → Processing → Analysis → Visualization → Output

Components

3.1 Data Sources
- Blizzard WoW Forums (primary live source)
- Local JSON datasets (manual or test data)
- Future: Reddit API integration

3.2 Collectors

Located in: `src/collectors/`

Responsibilities:
- Ingest raw data from sources
- Normalize into a unified schema

Key collectors:
- `file_collector.py` – loads local JSON datasets
- `blizzard_forum_collector.py` – auto-collects forum threads via JSON endpoints

---

4. Data Schema

All data is normalized into a consistent structure:

Field	Description
wow_class	Class name (e.g., Priest, Hunter)
spec	Specialization (e.g., Discipline, Beast Mastery)
source	Data source (e.g., blizzard_forums)
source_type	Type of record (thread, post)
author	Username
created_iso	Timestamp
score	Engagement metric (if available)
url	Source link
title	Thread title
body	Post content
combined_text	Title + body (used for analysis)

---

5. Processing Layer

Located in: `src/analysis/`

5.1 Sentiment Analysis (`sentiment.py`)
- Computes sentiment score from text
- Categorizes into:
  - Positive
  - Neutral
  - Negative

5.2 Theme Extraction (`themes.py`)

- Detects gameplay-related keywords
- Maps text to categories such as:
  - Damage
  - Survivability
  - Utility
  - Mobility

5.3 Role Classification (`summaries.py`)
- Maps specializations to roles:
  - Tank
  - Healer
  - DPS

---

6. Analysis Layer

6.1 Summary Tables

Generated via `build_summary_tables()`

Includes:
- Class sentiment summary
- Spec sentiment summary
- Role summary
- Theme frequency
- Top loved specs
- Most complained specs

6.2 Trend Analysis (`trends.py`)

Provides:
- Sentiment over time
- Spec popularity (mentions vs sentiment)
- Role vs theme matrix

6.3 Insights (`insights.py`)

Transforms raw analysis into readable insights:
- Identifies strongest and weakest specs

- Highlights role-level performance
- Detects major community concerns

6.4 Recommendations (`recommendations.py`)

Generates actionable outputs such as:
- Buff underperforming specs
- Address recurring negative themes
- Improve survivability or utility gaps

---

7. Visualization Layer

7.1 Excel Dashboard

Generated via `src/dashboard/excel_export.py`

Features:
- Multi-sheet report
- Embedded charts
- Summary tables
- Design insights and recommendations

7.2 Chart Generation (`charts.py`)

Uses Matplotlib to generate:
- Sentiment by class and spec
- Sentiment distribution
- Popularity vs sentiment scatter
- Role vs theme heatmap

7.3 Web Application (`app.py`)

Built using Streamlit and Plotly.

Features:
- Interactive filtering (class, spec, role, source)
- Real-time data exploration
- Interactive charts:

  - Line charts (time trends)
  - Scatter plots (popularity vs sentiment)
  - Heatmaps (role vs theme)
- Exportable datasets

---

8. Data Flow

1. Data is collected from forums or JSON input
2. Text is normalized into structured records
3. Sentiment and themes are extracted
4. Data is aggregated into summaries
5. Insights and recommendations are generated
6. Outputs are rendered as:
   - Excel dashboard
   - Web application

---

9. Key Features and Capabilities

- End-to-end analytics pipeline
- Multi-source data ingestion
- Automated insight generation
- Interactive visualization
- Scalable architecture for additional data sources
- Separation of concerns across modules

---

10. Limitations

- Spec detection relies on keyword inference (not fully contextual NLP)
- Forum data structure may change (requires parser updates)
- Sentiment analysis is rule-based, not model-based
- No real-time ingestion currently

---

11. Future Improvements

- Machine learning-based sentiment classification
- Improved spec detection using full-text analysis

- Integration with Reddit API or additional sources
- Patch/version tracking for longitudinal analysis
- Deployment as a hosted web application
- Enhanced UI/UX with advanced filtering and dashboards

---

12. Use Cases

This system can be applied to:

- Game balance analysis
- Player feedback aggregation
- Live service monitoring
- Design iteration support
- Community sentiment tracking

---

13. Conclusion

This project demonstrates how unstructured player feedback can be transformed into structured, actionable insights. By combining data processing, analytics, and visualization, it provides a scalable framework for incorporating community sentiment into game design decision-making.